```
public interface IEmailSender
{
    public void SendEmail(string username);
}
```

Then `RegisterUser` could depend on this interface instead of the specific `EmailSender` implementation, as shown in the following listing, allowing you to use a different implementation during unit tests, such as a `DummyEmailSender`.

Listing 8.5 Using interfaces with dependency injection

```
string RegisterUser(string username, IEmailSender emailSender)   ❶
{
    emailSender.SendEmail(username);                              ❷
    return $"Email sent to {username}!";
}
```

❶ Now you depend on IEmailSender instead of the specific EmailSender implementation.

❷ You don’t care what the implementation is as long as it implements IEmailSender.

The key point here is that the consuming code, `RegisterUser`, doesn’t care how the dependency is implemented—only that it implements the `IEmailSender` interface and exposes a `SendEmail` method. Now the application code is independent of the implementation.

I hope that the principles behind DI seem to be sound. Having loosely coupled code makes it easy to change or swap out implementations. But this still leaves a question: how does the application know to use `EmailSender` in production instead of `DummyEmailSender`? The process of telling your DI container “When you need `IEmailSender`, use `EmailSender`” is called *registration*.